Lekcja 10: Profesjonalna Praca z Git

`#lekcja` `#python` `#git` `#praca_zespolowa` `#conventional_commits` `#git_flow` `#repozytoria`

Cele lekcji

Po zakończeniu tej lekcji będziesz potrafił:

1. Efektywnie korzystać z Git w pracy zespołowej
2. Pisać czytelne commity zgodne z konwencją Conventional Commits
3. Stosować właściwe strategie tworzenia gałęzi
4. Przeprowadzać wysokiej jakości code review
5. Sprawnie korzystać z Git i rozumieć jego przepływ pracy
6. Wykorzystywać polecenia Git do zarządzania repozytorium
7. Pracować z platformami GitHub, GitLab i Bitbucket
8. Efektywnie współpracować nad kodem w środowisku zespołowym

1. Wprowadzenie do profesjonalnego Git: Podstawowe pojęcia i rozgałęzianie

W nowoczesnym świecie programowania Git nie jest już tylko narzędziem do przechowywania kodu, ale kluczowym elementem procesu tworzenia oprogramowania. Prawidłowe użycie Git może znacząco przyspieszyć pracę zespołu, poprawić jakość kodu i uczynić proces programowania bardziej przewidywalnym.

W celu rozwiązania problemów, programiści korzystają z systemu kontroli wersji Git. Jest to specjalny program, który pozwala kontrolować zmiany w plikach projektu zarówno w przypadku samodzielnego, jak i wspólnie tworzonego kodu. Podstawowe funkcje Gita obejmują:

- Powrót do dowolnej wersji kodu z przeszłości
- Przeglądanie historii zmian i przywracania dowolnych danych
- Możliwość współpracy bez obawy o utratę danych lub nadpisanie cudzej pracy

Aby naprawdę zrozumieć system kontroli wersji, musisz cofnąć się i przyjrzeć, jak Git przechowuje dane.

Kiedy tworzysz commit za pomocą polecenia `git commit`, Git oblicza sumy kontrolne dla każdego podkatalogu i przechowuje je w repozytorium jako obiekt drzewa katalogów. Następnie

Git tworzy obiekt commit z metadanymi i wskaźnikiem do głównego drzewa projektu, aby w razie potrzeby można było go odtworzyć.

Twoje repozytorium Git przechowuje teraz kilka obiektów: po jednym obiekcie typu blob dla każdego pliku, obiekt drzewa katalogów zawierający listę plików i odpowiadających im obiektów blob oraz obiekt commit zawierający metadane i wskaźnik do obiektu drzewa katalogów.

Kiedy commitujesz zmiany, Git przechowuje to jako obiekt zawierający wskaźnik do migawki zatwierdzonych danych. Obiekt ten zawiera także imię i nazwisko autora oraz adres e-mail, wiadomość i wskaźnik do zatwierdzenia lub zatwierdzeń bezpośrednio poprzedzających ten obiekt (jego rodziców): brak elementu nadrzędnego dla pierwotnego zatwierdzenia, jeden element nadrzędny dla zwykłego zatwierdzenia i wielu rodziców dla wyników połączenia dwóch lub więcej oddziałów.

```
commit abc123
Author: Jan Kowalski <jan.kowalski@example.com>
Date:    2025-02-07
Message: "Initial commit"
Tree: def456 (wskaźnik na obiekt drzewa)
```

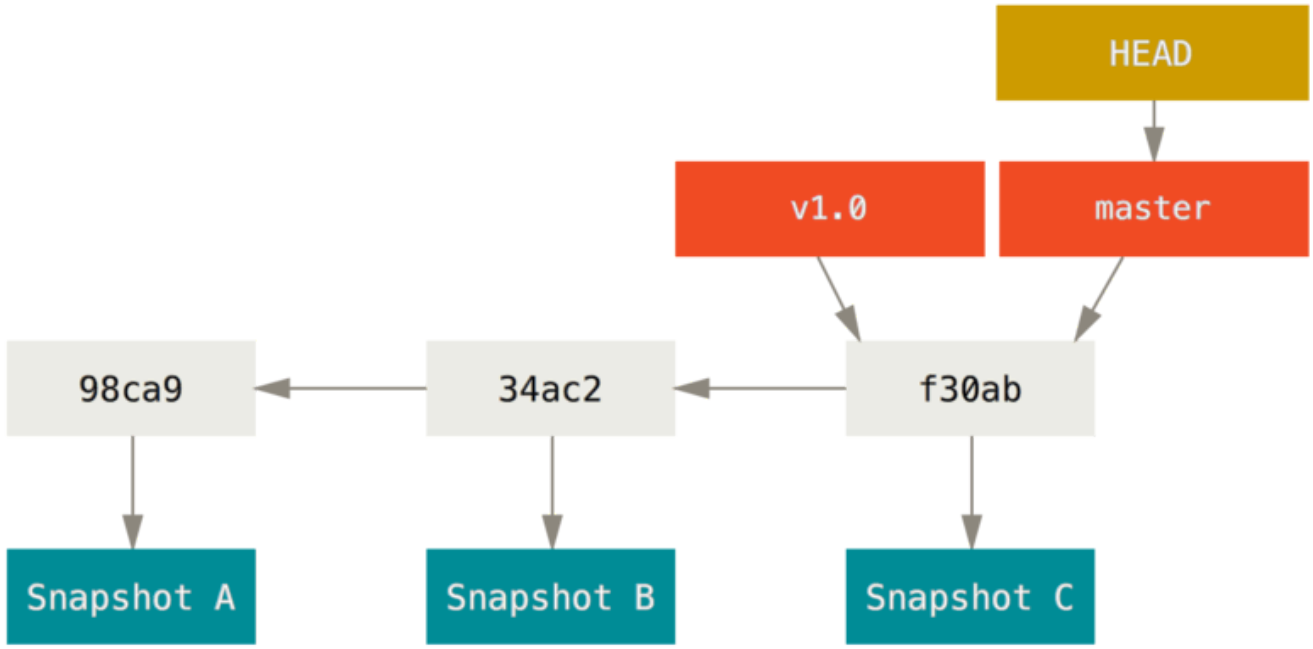
1.1 Historia Git i praca zespołowa

Przed pojawieniem się systemów kontroli wersji zespoły programistów zmagały się z wieloma problemami:

- Konflikty podczas jednoczesnej pracy
- Trudności z tworzeniem i scalaniem gałęzi
- Problemy z pracą offline

> **Notka historyczna**
>
> Linus Torvalds stworzył Git w 2005 roku na potrzeby rozwoju jądra Linux, kładąc fundamenty, które dziś są standardem w branży:
>
> - Architektura rozproszona
> - Szybka praca z gałęziami
> - Niezawodność przechowywania danych

1.2 Historia rozgałęzień i zatwierdzeń:



2. Podstawowe polecenia Gita

2.1 Konfiguracja użytkownika w Git

Aby ustawić lub zmienić nazwę użytkownika, użyj polecenia:

```
git config --global user.name "Gal Anonim"
```

Nowa nazwa pojawi się w kolejnych commitach przesyłanych do GitHub. Możesz użyć pseudonimu, jeśli nie chcesz ujawniać swojego prawdziwego imienia i nazwiska.

2.2 Inicjalizacja repozytorium

Możesz utworzyć nowe repozytorium lub ponownie zainicjować istniejące:

```
git init
```

To polecenie tworzy ukryty katalog `.git`, który przechowuje wszystkie dane dotyczące historii projektu.

Wynik:

```
Initialized empty Git repository in /path/to/repo/.git/
```

2.3 Dodawanie plików do obszaru staging

Aby dodać pojedynczy plik do obszaru staging (przygotowania do commitowania):

```
git add somefile.js
```

Możesz dodać wszystkie zmodyfikowane pliki naraz:

```
git add .
```

2.4 Sprawdzanie statusu repozytorium

Aby zobaczyć status plików (czy są śledzone, zmodyfikowane, gotowe do commitowania):

```
git status
```

Wynik jeśli nie ma zmian:

```
On branch main nothing to commit, working tree clean
```

Wynik jeśli są nieśledzone pliki:

```
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        newfile.txt
nothing added to commit but untracked files present (use "git add" to track)
```

2.5 Tworzenie commitu

Aby zatwierdzić zmiany z krótkim opisem:

```
git commit -m "Krótki opis zmian"
```

Jeśli chcesz dodać bardziej szczegółowy opis, uruchom:

```
git commit
```

To otworzy edytor tekstowy w terminalu, gdzie możesz wpisać dłuższą wiadomość.

Wynik utworzenia commita:

```
[main e5f6d7c] Dodano nowy plik
 1 file changed, 1 insertion(+)
 create mode 100644 newfile.txt
```

2.6 Wyświetlanie historii commitów

Aby zobaczyć historię commitów w repozytorium:

```
git log
```

Wynik:

```
commit b1c2cd258e1ee273febb7227489ad3a1a4e5a365 (HEAD -> master)
Author: Gal Anonim <gal.anonim@learnit.com.pl>
Date:   Wed Feb 12 17:14:13 2025 +0000

    Dodano nowy plik2

commit 47a30456bc7b65d4326a2846f25008497b168deb
Author: Gal Anonim <gal.anonim@learnit.com.pl>
Date:   Wed Feb 12 17:12:49 2025 +0000

    Dodano nowy plik
```

Aby zobaczyć szczegóły zmian w commitach:

```
git log -p
```

Wynik:

```
commit b1c2cd258e1ee273febb7227489ad3a1a4e5a365 (HEAD -> master)
Author: Gal Anonim <gal.anonim@learnit.com.pl>
Date:   Wed Feb 12 17:14:13 2025 +0000

    Dodano nowy plik2

diff --git a/test2 b/test2
new file mode 100644
index 0000000..e69de29

commit 47a30456bc7b65d4326a2846f25008497b168deb
Author: Gal Anonim <gal.anonim@learnit.com.pl>
```

```
Date:   Wed Feb 12 17:12:49 2025 +0000

    Dodano nowy plik

diff --git a/test b/test
new file mode 100644
index 0000000..e69de29
```

2.7 Usuwanie plików z repozytorium

Aby usunąć plik zarówno z katalogu roboczego, jak i repozytorium:

```
git rm somefile.js
```

Możesz także użyć masek plików, aby usunąć wszystkie pliki spełniające określone kryteria:

```
git rm *.html
```

2.8 Tworzenie i przełączanie gałęzi

Aby utworzyć nową gałąź:

```
git branch nowa_galaz
```

Aby przełączyć się na nową gałąź:

```
git checkout nowa_galaz
```

Wynik:

```
Switched to branch 'nowa_galaz'
```

Możesz również utworzyć i od razu przełączyć się na nową gałąź:

```
git checkout -b nowa_galaz
```

Wynik:

```
Switched to branch 'nowa-galaz'
```

2.9 Scalanie gałęzi

Aby połączyć dwie gałęzie, użyj polecenia `merge`:

```
git merge nazwa_istniejacej_galezi
```

Wynik:

```
Updating e5f6d7c..8a9b0c1
Fast-forward
 newfile.txt | 1 +
 1 file changed, 1 insertion(+)
```

Jeśli chcesz zachować historię commitów bez szybkiego łączenia (`fast-forward`):

```
git merge --no-ff nazwa_istniejacej_galezi
```

Wynik:

```
Merge made by the 'ort' strategy.
 test4 | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test4
```

2.10 Wysyłanie zmian do zdalnego repozytorium

Aby wysłać zmiany do zdalnego repozytorium (np. GitHub):

```
git push origin main
```

To polecenie wypycha lokalne zmiany do centralnego repozytorium.

Wynik:

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 345 bytes | 345.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0)
```

```
To github.com:user/repo.git
   b2a1c3d..e5f6d7c  main -> main
```

2.11 Pobieranie zmian ze zdalnego repozytorium

Aby pobrać zmiany i połączyć je z lokalną kopią repozytorium:

```
git pull
```

Wynik nie ma zmian:

```
Already up to date.
```

Wynik jeśli są zmiany:

```
From github.com:user/repo
 * branch          main      -> FETCH_HEAD
Updating e5f6d7c..a8b9c0d
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Aby zobaczyć szczegóły pobranych plików, użyj flagi `--verbose`:

```
git pull --verbose
```

Wynik jeśli nie ma zmian:

```
POST git-upload-pack (223 bytes)
From github.com:user/repo
 = [up to date]      main          -> origin/main
 = [up to date]      feature/new-feature -> origin/feature/new-feature
Already up to date.
```

Wynik jeśli są zmiany:

```
POST git-upload-pack (223 bytes)
From github.com:user/repo
 = [up to date]          main          -> origin/main
 * [new branch]          fix/bug_tooltips    -> origin/fix/bug_tooltips
```

```
 = [up to date]          feature/new-feature -> origin/feature/new-feature
   a949dc958..01de67f4f  develop         -> origin/develop
```

3. Zaawansowane polecenia Git

3.1 Rebasing (Zmiana bazy commitów)

> **Definition**
>
> Rebasing to proces przenoszenia lub łączenia sekwencji commitów do nowej bazy. Pozwala to na stworzenie bardziej liniowej i czystej historii projektu, eliminując zbędne commity scalające.

Aby zmienić punkt początkowy commitów:

```
git rebase nazwa_galezi
```

To polecenie zmieni bazę gałęzi z jednego commita na drugi, reorganizując ich historię.

Wynik:

```
Successfully rebased and updated refs/heads/dev.
```

3.2 Podgląd konkretnego commitu

Aby podejrzeć szczegóły konkretnego commitu:

```
git show 1a2b3c4d
```

To polecenie pokaże szczegóły commitu o podanym identyfikatorze (hashu commita).

Wynik:

```
commit e5f6d7c1a2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p
Author: Gal Anonim <gal@learnit.com.pl>
Date:   Sat Feb 9 12:34:56 2025 +0100

    Dodano nowy plik

diff --git a/newfile.txt b/newfile.txt
new file mode 100644
```

```
index 0000000..e69de29
diff --git a/newfile2.txt b/newfile2.txt
new file mode 100644
index 0000000..e69de29
```

3.3 Wyświetlenie zmian tylko dla konkretnego pliku

Aby podejrzeć zmiany konkretnego pliku w danym commicie:

```
git show 1a2b3c4d -- somefile.js
```

Dzięki temu zobaczysz tylko zmiany wprowadzone w `somefile.js` w danym commicie.

Wynik:

```
commit e5f6d7c1a2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p
Author: Gal Anonim <gal@learnit.com.pl>
Date:   Sat Feb 9 12:34:56 2025 +0100

    Dodano nowy plik

diff --git a/newfile.txt b/newfile.txt
new file mode 100644
index 0000000..e69de29
```

3.4 Krótszy format wyjścia (bez różnic w plikach)

Aby podejrzeć skrócone dane commitu:

```
git show 1a2b3c4d --no-patch
```

To pokaże tylko metadane commitu i wiadomość, bez zmian w kodzie.

Wynik:

```
commit e5f6d7c1a2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p
Author: Gal Anonim <gal@learnit.com.pl>
Date:   Sat Feb 9 12:34:56 2025 +0100

    Dodano nowy plik
```

4. Główne zasady pracy z Git i strategie tworzenia gałęzi

4.1 Atomowość commitów

Definition

Atomowy commit to pojedyncza, spójna zmiana, która rozwiązuje jeden problem lub dodaje jedną funkcjonalność. Każdy commit powinien być na tyle mały, aby był łatwy do zrozumienia i wycofania, ale jednocześnie na tyle duży, aby był samodzielną, działającą zmianą.

Zły przykład:

```
git commit -m "Fixed bugs, updated styles, added new feature"
```

Dlaczego to jest złe?

- Nie da się zrozumieć, co dokładnie się zmieniło.
- Trudno cofnąć konkretne zmiany bez wpływu na inne.
- Utrudnia lokalizację błędów, ponieważ wiele zmian jest połączonych.

Dobry przykład:

```
git commit -m "fix: resolve memory leak in user service"
git commit -m "style: update button colors to match design system"
git commit -m "feat: add password reset functionality"
```

4.2 Conventional Commits

Definition

Conventional Commits to lekka konwencja dotycząca komunikatów commitów. Dostarcza ona zestaw prostych zasad tworzenia jawnej historii commitów, co ułatwia automatyzację narzędzi, takich jak generowanie changelogów czy automatyczne wersjonowanie.

Struktura commitów

```
<type>(<scope>): <description>

[opcjonalne body]
```

```
[opcjonalne stopki]
```

- **`<type>`** : Obowiązkowy typ commita, np. `feat`, `fix`, `docs`.
- **`(<scope>)`** : Opcjonalny zakres, który może wskazywać na część bazy kodu, której dotyczy zmiana (np. `auth`, `payments`, `api`).
- **`<description>`** : Krótki, zwięzły opis zmiany.

Typy commitów:

- `feat` : nowa funkcjonalność

  ```
  feat(auth): add Google OAuth authentication
  ```

- `fix` : poprawka błędu

  ```
  fix(payments): resolve double-charging issue
  ```

- `docs` : zmiany w dokumentacji

  ```
  docs(api): update authentication endpoints description
  ```

Praktyczna wskazówka

Używaj git commit bez flagi -m, aby pisać szczegółowe komunikaty. Otworzy się edytor tekstowy (np. Vim, Nano), gdzie możesz napisać pierwszą linię (tytuł commita), a następnie puste linie i szczegółowy opis (body).

```
git commit
# Otworzy się edytor, gdzie możesz napisać szczegółowy komunikat
```

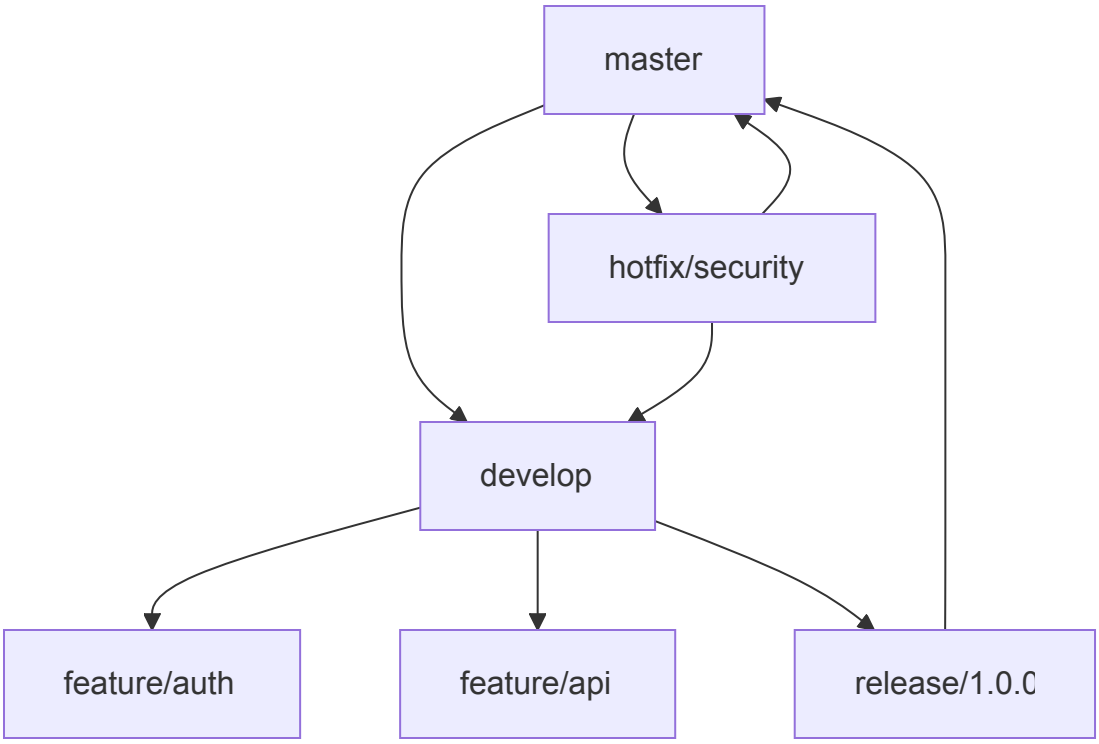
4.3 Strategie tworzenia gałęzi

Definition

Strategia tworzenia gałęzi (Branching Strategy) to zbiór zasad i konwencji, które zespół programistów stosuje do zarządzania gałęziami w repozytorium Git. Ma to na celu zapewnienie uporządkowanego przepływu pracy, ułatwienie współpracy i zarządzanie wydaniami.

Git Flow

Git-flow to popularny model rozgałęzień, który wprowadza uporządkowany sposób zarządzania kodem. Wykorzystuje gałęzie funkcji oraz wiele gałęzi głównych, dzięki czemu umożliwia równoczesną pracę nad nowymi funkcjami i stabilnym kodem produkcyjnym. Model ten został spopularyzowany przez Vincenta Driessena.



Schemat przedstawiający główne gałęzie i ich relacje w modelu Git Flow.

Główne gałęzie w Git Flow:

1. **master (main)**

   > **Info**
   > Gałąź master (lub main) zawsze reprezentuje stabilną, gotową do produkcji wersję kodu. Commity są do niej dodawane tylko poprzez scalanie gałęzi release lub hotfix.

   ```
   # Przykład tworzenia tagu wydania na gałęzi master
   git tag -a v1.0.0 -m "Release version 1.0.0"
   ```

2. **develop**

   > **Info**
   > Gałąź develop służy do integracji wszystkich nowych funkcjonalności. Jest to gałąź, do której scalane są gałęzie feature po ich ukończeniu.

```
git checkout -b develop
git push -u origin develop
```

3. **feature branches (gałęzie funkcjonalności)**

> **Info**
> Gałęzie feature są tworzone z gałęzi develop dla każdej nowej funkcjonalności. Po zakończeniu pracy nad funkcją, są one scalane z powrotem do develop.

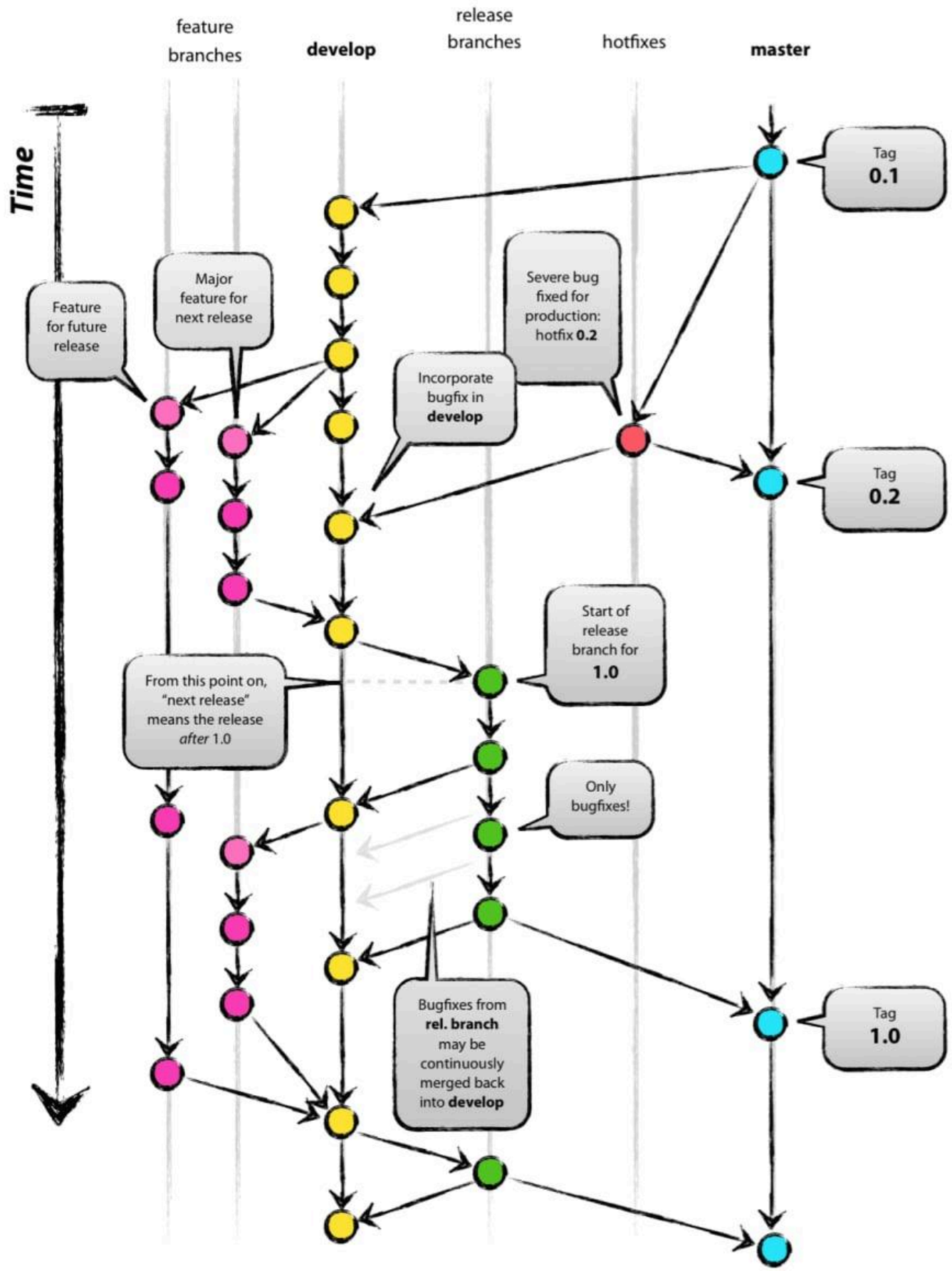
```
# Tworzenie nowej funkcjonalności za pomocą narzędzia git-flow
git flow feature start user-auth

# Lub bez git-flow (ręcznie)
git checkout -b feature/user-auth develop
```

4.4 Przepływ Gita

Git-flow to alternatywny model rozgałęzień w Git, który wprowadza uporządkowany sposób zarządzania kodem. Wykorzystuje gałęzie funkcji oraz wiele gałęzi głównych, dzięki czemu umożliwia równoczesną pracę nad nowymi funkcjami i stabilnym kodem produkcyjnym. Model ten został spopularyzowany przez Vincenta Driessena.

W przeciwieństwie do trunk-based development, Git-flow opiera się na większej liczbie gałęzi, które istnieją dłużej, a commity są zwykle większe. Programiści tworzą gałęzie funkcji, które łączą z główną linią dopiero po ich ukończeniu, co zapewnia lepszą kontrolę nad wersjami i wdrożeniami.

**



4.5 Przepływ pracy Git-flow w teorii:

1. **Inicjalizacja repozytorium** – tworzone jest repozytorium Git i rozpoczyna się jego konfiguracja.
2. **Tworzenie gałęzi głównych** – zazwyczaj mamy dwie główne gałęzie:
   - `main` (stabilna wersja produkcyjna)
   - `develop` (wersja rozwojowa)
3. **Rozpoczęcie nowej funkcji** – programista tworzy **gałąź funkcji** (`feature`), aby pracować nad nową zmianą.
4. **Praca nad funkcją** – w gałęzi funkcji wykonywane są commity i testy.
5. **Scalanie funkcji** – po zakończeniu pracy gałąź funkcji jest łączona z `develop`.
6. **Przygotowanie wersji do wydania** – gdy `develop` osiągnie stabilny stan, tworzona jest **gałąź wydania** (`release`). W niej można poprawiać błędy i optymalizować kod.
7. **Publikacja wersji** – po zakończeniu testów **gałąź wydania jest scalana z `main`**, a jej zmiany są również przenoszone do `develop`.
8. **Obsługa poprawek** – jeśli w `main` wykryto błąd, tworzy się **gałąź hotfix**, która po poprawieniu problemu jest scalana zarówno z `main`, jak i `develop`.

5. Recenzja usług GitHub, GitLab, Bitbucket

5.1 GitHub

GitHub, Inc. to platforma należąca do Microsoftu, która zapewnia hosting do tworzenia oprogramowania i kontroli wersji za pomocą Git. Oferuje funkcje takie jak:

- Rozproszona kontrola wersji Git
- Śledzenie problemów i zarządzanie projektami
- Pull requesty do przeglądania i zatwierdzania zmian
- CI/CD (ciągła integracja i dostarczanie kodu)
- Wiki i dokumentacja projektów

Siedziba GitHuba znajduje się w Kalifornii, a od 2018 roku jest własnością Microsoftu.

5.2 GitLab

GitLab to platforma DevOps, która oprócz hostowania kodu źródłowego, zapewnia:

- Repozytoria Git
- Wiki i system śledzenia problemów
- Zaawansowane funkcje CI/CD
- Narzędzia do zarządzania zespołami i projektami

GitLab został stworzony przez Dmitrija Zaporozhetsa i Valerego Sizova i działa na licencji open-source.

5.3 Bitbucket

Bitbucket to usługa hostingu repozytoriów Git i Mercurial, należąca do Atlassian. Oferuje:

- Integrację z Jira i innymi narzędziami Atlassian
- Nieograniczoną liczbę prywatnych repozytoriów (do 5 użytkowników w darmowej wersji)
- Wsparcie dla CI/CD
- Możliwość hostowania kodu w chmurze lub na własnych serwerach

Bitbucket jest popularny wśród zespołów korzystających z narzędzi Atlassian i oferuje zarówno płatne, jak i darmowe plany.

🧪 Zadania do samodzielnej pracy

Zadania Proste (Easy)

1. ✏️ **Zadanie 1 – Inicjalizacja repozytorium i pierwszy commit**

   1. Zainicjalizuj nowe repozytorium Git w dowolnym katalogu na swoim komputerze.
   2. Stwórz plik `README.md` i dodaj do niego krótki opis projektu.
   3. Dodaj plik do obszaru staging i zatwierdź zmiany z odpowiednim komunikatem commit.
   4. Wyświetl historię commitów za pomocą `git log`.

   Oczekiwany rezultat: Wykonanie poleceń git init, git add, git commit i git log poprawnie wyświetli historię commitów.

   (proste)
2. ✏️ **Zadanie 2 – Praca z gałęziami**

   1. Utwórz nową gałąź o nazwie `feature-login`.
   2. Przełącz się na nową gałąź i dodaj nowy plik `login.py` (lub `login.html` jeśli wolisz).
   3. Zatwierdź zmiany w tej gałęzi i przełącz się z powrotem na `main`.
   4. Spróbuj połączyć gałąź `feature-login` z `main` i rozwiąż ewentualne konflikty.

   Oczekiwany rezultat: Powstanie nowej gałęzi, zatwierdzenie zmian, scalanie bez błędów lub rozwiązanie konfliktów.

   (proste)
3. ✏️ **Zadanie 3 – Klonowanie i współpraca**

1. Sklonuj podane na zajęciach repozytorium z GitHub (lub stwórz własne puste repozytorium na GitHub i sklonuj je).
2. Dodaj nowy plik `contributors.txt`, wpisz w nim swoje imię i nazwisko.
3. Stwórz nową gałąź, zatwierdź zmiany i wypchnij je do zdalnego repozytorium.
4. Utwórz Pull Request (lub Merge Request) na platformie GitHub/GitLab/Bitbucket i opisz zmiany.

Oczekiwany rezultat: Klonowanie repozytorium, dodanie pliku, utworzenie PR/MR.

(proste)

4. ✏️ **Zadanie 4 – Konfiguracja projektu (podstawy)**

   1. Utwórz nową repozytorię:

      ```
      mkdir team-project-basic
      cd team-project-basic
      git init
      ```

   2. Skonfiguruj swoje dane użytkownika dla tej repozytorium:

      ```
      git config user.name "Twoje Imię"
      git config user.email "twoj.email@example.com"
      ```

(proste)

5. ✏️ **Zadanie 5 – Praca z plikami i commitami**
   1. W repozytorium `team-project-basic` (z Zadania 4) stwórz plik `app.py` z prostym kodem Pythona (np. funkcja `hello_world`).
   2. Dodaj `app.py` do obszaru staging.
   3. Wykonaj commit z komunikatem zgodnym z Conventional Commits (np. `feat: add hello world function`).
   4. Zmodyfikuj `app.py` (np. zmień komunikat funkcji).
   5. Sprawdź status repozytorium (`git status`).
   6. Dodaj zmieniony plik do stagingu i wykonaj kolejny commit (np. fix: update hello world message).

      (proste)

Zadania Trudniejsze (Challenge)

6. 🧠 **Zadanie 6 – Resetowanie i przywracanie zmian**

   1. Wprowadź kilka zmian w istniejącym pliku i dodaj je do stagingu (`git add`).
   2. Wycofaj te zmiany z obszaru staging (`git reset HEAD <plik>`).

3. Wprowadź kolejne zmiany w tym samym pliku i wykonaj commit.
4. Cofnij się do commita sprzed tych ostatnich zmian za pomocą `git reset --hard <hash_commita>`. Upewnij się, że rozumiesz, co robi `--hard`.

Oczekiwany rezultat: Zmiany poprawnie cofnięte lub przywrócone do wcześniejszego stanu.

(challenge)

7. 🧠 **Zadanie 7 – Rebase i interaktywna edycja historii**

   1. Utwórz nową gałąź `feature-branch` z `main`.
   2. W gałęzi `feature-branch` wykonaj trzy oddzielne commity, każdy z małą, prostą zmianą (np. dodaj jedną linię do pliku, zmień drugą, dodaj komentarz).
   3. Użyj `git rebase -i HEAD~3`, aby połączyć te trzy commity w jeden spójny commit. Zmień typ commita na `feat` i użyj konwencji Conventional Commits.
   4. Wypchnij zmiany do zdalnego repozytorium (jeśli wymagane, użyj `git push --force` – pamiętaj, że to nadpisuje historię!).

   Oczekiwany rezultat: Historia commitów została poprawnie zmieniona i wypchnięta.

   (challenge)

8. 🧠 **Zadanie 8 – Rozwiązywanie konfliktów w Git**

   1. **Utwórz repozytorium lokalnie** i zainicjalizuj Git (`git init`).
   2. **Dodaj plik `conflict_example.txt`**, wpisz do niego pierwszą linię tekstu (np. `Linia 1 - wersja glowna`) i wykonaj commit (`git commit -m "feat: dodano pierwsza linie"`).
   3. **Utwórz nową gałąź `branch-A`** (`git checkout -b branch-A`) i edytuj `conflict_example.txt`, dodając drugą linię (np. `Linia 2 - z branch-A`). Zatwierdź zmiany (`git commit -m "feat: dodano druga linie w branch-A"`).
   4. **Przełącz się na `main`** (`git checkout main`) i również edytuj `conflict_example.txt`, ale w inny sposób (np. zmień pierwszą linię na `Linia 1 - zmieniona w main` i dodaj nową linię `Linia 2 - z main`). Zatwierdź zmiany (`git commit -m "fix: zmieniono pierwsza linie i dodano druga w main"`).
   5. **Spróbuj scalić `branch-A` z `main`** (`git merge branch-A`). Zauważysz konflikt.
   6. **Rozwiąż powstały konflikt** w pliku `conflict_example.txt`, edytując go ręcznie, a następnie dodaj rozwiązany plik do stagingu (`git add conflict_example.txt`).
   7. **Zatwierdź poprawioną wersję pliku** i zakończ scalanie (`git commit`).

   Oczekiwany rezultat: Poprawnie rozwiązany konflikt merge w pliku conflict_example.txt.

   (challenge)

9. 🧠 Zadanie 9 – Zastosowanie Conventional Commits (zaawansowane)

W ramach dowolnego z poprzednich zadań, upewnij się, że wszystkie Twoje commity są zgodne z konwencją Conventional Commits. Stosuj typy takie jak feat, fix, docs, style i dodawaj zakres, jeśli to ma sens. Dodatkowo, użyj opcjonalnego body (szczegółowego opisu) dla co najmniej jednego commita, wyjaśniając, dlaczego zmiana została wprowadzona.

(challenge)

10. 🧠 Zadanie 10 – Praca z Git Flow (zaawansowane)

    Jeśli masz zainstalowane narzędzie git-flow (lub jesteś gotów symulować jego działanie ręcznymi komendami git branch/git checkout/git merge):

    1. Zainicjalizuj `git flow` w nowym repozytorium.
    2. Rozpocznij nową funkcjonalność (`git flow feature start moja-funkcja`).
    3. Wprowadź kilka commitów na tej gałęzi.
    4. Zakończ funkcjonalność (`git flow feature finish moja-funkcja`).
    5. Zauważ, jak zmiany zostały scalone z gałęzią `develop` .
    6. Następnie, rozpocznij gałąź wydania (`git flow release start 1.0.0`).
    7. Wprowadź "poprawkę" w gałęzi wydania (np. zmień coś w pliku i zatwierdź).
    8. Zakończ wydanie (`git flow release finish 1.0.0`). Zauważ, jak wydanie zostało scalone zarówno z `main` , jak i `develop` , a poprawka jest widoczna w obu gałęziach.

    Oczekiwany rezultat: Zrozumienie i praktyczne zastosowanie cyklu życia gałęzi w Git Flow.

    (challenge)

🏠 Zadania domowe

Zadanie domowe 1: Zespołowe tworzenie małego projektu

1. **Przygotowanie projektu (1 godzina)**

    1. **Tworzenie repozytorium:**
        - [ ] Utwórz nową repozytorię na GitHub/GitLab.
        - [ ] Skonfiguruj ochronę gałęzi dla gałęzi `main` (np. wymagaj 2 recenzentów, wymagaj przejścia testów, zablokuj bezpośrednie commity).
        - [ ] Dodaj pliki `.gitignore` (dla Pythona) i `README.md` (z opisem projektu).
        - [ ] Skonfiguruj Git Flow (jeśli używasz narzędzia `git-flow` lub stwórz gałąź `develop` ręcznie).
    2. **Konfiguracja środowiska:**

    ```
    # Klonowanie repozytorium
    git clone https://pl.wikipedia.org/wiki/Repozytorium # Zastąp to
    ```

```
rzeczywistym URL repozytorium
cd [nazwa projektu]

# Inicjalizacja Git Flow (jeśli nie zrobiono wcześniej)
# git flow init

# Tworzenie podstawowej struktury
mkdir src tests docs
```

(challenge)

Zadanie domowe 2: Tworzenie i zarządzanie repozytorium na GitHub

1. **Utwórz nowe repozytorium** na GitHub (lub GitLab/Bitbucket) i sklonuj je na swój komputer.
2. **Dodaj plik `projekt.md`** i opisz w nim swój pomysł na mały projekt (np. aplikację, stronę internetową).
3. **Stwórz nową gałąź `feature-readme`**, wprowadź zmiany w pliku `projekt.md` i zatwierdź je, używając konwencji Conventional Commits (np. `docs: dodano opis projektu`).
4. **Wypchnij zmiany do zdalnego repozytorium** i utwórz Pull Request na GitHubie. Upewnij się, że Pull Request jest czytelny i zawiera odpowiedni opis.
5. **Zmień główną gałąź (`main` lub `master`)** i scal Pull Request (możesz to zrobić z poziomu interfejsu GitHub).
6. **Usuń lokalnie i zdalnie gałąź `feature-readme`**, ponieważ nie jest już potrzebna.

Oczekiwany rezultat: Repozytorium z poprawnie zarządzaną historią commitów i scalonym PR.

(challenge)

Zadanie domowe 3: Rozwiązywanie konfliktów w Git

1. **Utwórz repozytorium lokalnie** i zainicjalizuj Git (`git init`).
2. **Dodaj plik `konflikt.txt`**, wpisz do niego pierwszą linię tekstu (np. "Pierwsza linia tekstu.") i wykonaj commit (`git commit -m "feat: dodano poczatkowy tekst"`).
3. **Utwórz nową gałąź `galaz-A`** (`git checkout -b galaz-A`) i edytuj `konflikt.txt`, dodając nową linię w środku pliku (np. "Linia dodana w galaz-A."). Zatwierdź zmiany (`git commit -m "feat: dodano linie z galaz-A"`).
4. **Przełącz się na `main`** (`git checkout main`) i również edytuj `konflikt.txt`, dodając inną nową linię w tym samym miejscu w pliku (np. "Linia dodana w main."). Zatwierdź zmiany (`git commit -m "feat: dodano linie z main"`).
5. **Spróbuj scalić `galaz-A` z `main`** (`git merge galaz-A`). Zauważysz konflikt.

6. **Rozwiąż powstały konflikt** w pliku `konflikt.txt`, edytując go ręcznie tak, aby zawierał obie dodane linie (np. "Pierwsza linia tekstu.\nLinia dodana w galaz-A.\nLinia dodana w main."). Następnie dodaj rozwiązany plik do stagingu (`git add konflikt.txt`).
7. **Zatwierdź poprawioną wersję pliku** i zakończ scalanie (`git commit -m "merge: rozwiazano konflikt w konflikt.txt"`).

Oczekiwany rezultat: Poprawnie rozwiązany konflikt merge w pliku konflikt.txt.

(challenge)

Kryteria oceny

Poziom podstawowy (obowiązkowy):

- [ ] Repozytoria zostały poprawnie utworzone i skonfigurowane.
- [ ] Wykonano podstawowe operacje w Git (np. dodanie pliku, commit, przełączanie gałęzi).
- [ ] Zmiany zostały poprawnie zapisane i wypchnięte do repozytorium (w przypadku pracy ze zdalnym repozytorium).
- [ ] Poprawne użycie Git Flow (jeśli było wymagane/wybrane).
- [ ] Stosowanie Conventional Commits dla większości commitów.
- [ ] Wysokiej jakości Pull Requests (czytelne opisy, prawidłowe gałęzie).

Poziom zaawansowany:

- [ ] Poprawnie wykorzystano gałęzie (np. utworzenie gałęzi funkcji, rozwiązanie konfliktu, scalanie zmian).
- [ ] Historia commitów jest czytelna i dobrze zorganizowana (brak zbędnych commitów, spójne opisy commitów, użycie `rebase -i`).
- [ ] Repozytorium jest uporządkowane (np. usunięcie niepotrzebnych gałęzi, poprawna struktura plików, `.gitignore`).
- [ ] Automatyzacja sprawdzeń (jeśli została zaimplementowana, np. pre-commit hooki z `commitlint` / `husky`).
- [ ] Wdrożenie CI/CD (jeśli zostało zaimplementowane i zademonstrowane).
- [ ] Dokumentacja procesów (jeśli została stworzona).
- [ ] Zarządzanie wydaniami (jeśli zostało zaimplementowane i zademonstrowane, np. tagowanie wydań).

Dodatkowe materiały

📚 Polecana literatura

1. [Oficjalna dokumentacja Gita]

2. **Dokumentacja GitHub**
3. **Dokumentacja GitLab**
4. Pro Git Book
5. Conventional Commits Specification
6. Git Flow Cheat Sheet

🛠️ Przydatne narzędzia

1. Git Flow (narzędzie wiersza poleceń)
2. commitlint (narzędzie do walidacji komunikatów commitów)
3. husky (narzędzie do zarządzania git hookami)
4. conventional-changelog (narzędzie do generowania changelogów)
5. Git Kraken (graficzny klient Git)
6. GitHub Desktop (graficzny klient Git)
7. gitk (prosty graficzny przeglądarka historii Git)